UPSKILL HUB

A

SYNOPSIS REPORT OF PROJECT

Upskill Hub

For the course **FRONT END WEB DEVELOPMENT (26CA205PCB)**

In partial fulfillment of requirements for the degree of

MASTER OF COMPUTER APPLICATIONS (MCA)

SUBMITTED BY:

Utkarsh Jain 2628MCA0128
Uttam Kumar 2628MCA0194
Vanshika Tyagi 2628MCA0066

August 2026



SUBMITTED TO: Mr. SHASHANK SIR

MASTER OF COMPUTER APPLICATIONS (MCA)
Department of Computer Applications
Krishna Institute of Engineering & Technology
(KIET),
Ghaziabad, Delhi-NCR, Uttar Pradesh, India

➤ Contents

1. Abstract

2. Motivation

3. Introduction

   o Objectives

   o Future Scope

   o Findings

4. Analysis Phase

   o Requirement Analysis

   o Data Flow Diagram (DFD) / Use Case Diagram

   o System Flow (if applicable)

5. Technology Stack (Web Development)

6. System Design & Methodology

7. Expected Outcomes (Website / Web App)

8. References

➤ Abstract

Upskill Hub is a responsive, multi-page front-end web application designed to serve as an online learning and course-discovery platform. The project presents a clean, modular website where learners can browse a curated catalog of programming courses, view instructor and community-expert profiles, read educational blog articles, register for free trial access, and get in touch with the platform through a dedicated contact page.

Built entirely with HTML5, CSS3, and vanilla JavaScript, the platform uses the Fetch API to dynamically load reusable navigation and footer components across every page, keeping the codebase modular and easy to maintain. The interface follows a mobile-first, responsive design approach, complete with an animated countdown-based registration section, star-rated course cards, and social-linked expert profiles. The project demonstrates strong command of semantic HTML structuring, custom CSS styling, component-based front-end architecture, and DOM manipulation — laying a solid foundation for future backend and database integration.

➤ Motivation

The e-learning industry has grown rapidly as students and working professionals increasingly turn to online platforms to acquire new technical skills at their own pace. However, many independent learners struggle to find a single, well-organized destination that presents course offerings, pricing, instructor credibility, and learning resources together in one place.

Upskill Hub was motivated by the need to design a clean, trustworthy, and easy-to-navigate front-end experience that mirrors the structure of real-world ed-tech platforms. By focusing purely on the front-end layer first, the project allowed the team to concentrate on usability, visual hierarchy, and responsive behaviour before layering in backend complexity.

From a technical perspective, this project demonstrates expertise in:

- Building a multi-page website with consistent, reusable header and footer components
- Structuring semantic, accessible HTML across distinct page types (home, about, blog, courses, contact)
- Writing custom, from-scratch CSS for a fully responsive grid and flexbox-based layout
- Using the Fetch API and DOM manipulation for dynamic component loading and mobile-menu interactivity
- Organizing static assets (images, icons) and stylesheets for a scalable front-end codebase

The platform serves as a practical front-end case study, with a clear, incremental pathway toward becoming a fully dynamic learning management system.

➤ Introduction

3.1 Domain Overview

The online education domain has seen a sharp rise in demand as learners seek flexible, self-paced ways to build in-demand technical skills. Upskill Hub operates in this space as a course-discovery and learning-promotion platform, showcasing six popular programming courses — JavaScript, CSS, Python for Beginners, C++, Java, and AngularJS — alongside scholarship facilities, global certification messaging, and a community of subject-matter experts.

3.2 Problem Statement

Independent learners and small ed-tech initiatives traditionally face three recurring challenges:

- **Fragmented Course Discovery:** Course details, pricing, and ratings are often scattered across multiple pages or platforms, making comparison difficult for learners.
- **Inconsistent Navigation Experience:** Without reusable, centrally maintained navigation and footer components, multi-page educational sites become inconsistent and hard to maintain.
- **Poor Mobile Accessibility:** Many learning websites are not fully optimised for mobile devices, leading to a frustrating experience for on-the-go learners.

3.3 Proposed Solution

Upskill Hub solves these problems through:

- **Centralized Course Catalog:** All courses are displayed with thumbnails, star ratings, update dates, and clear pricing on both the home page and a dedicated courses page.
- **Reusable Component Architecture:** The navigation bar and footer are built as standalone HTML fragments and dynamically injected into every page via JavaScript's Fetch API, ensuring consistency and easy maintenance.
- **Community & Trust Building:** Dedicated sections for community experts and learner testimonials help build credibility for the platform.
- **Responsive, Mobile-First Design:** A collapsible mobile navigation menu and fluid CSS layouts ensure the site works seamlessly across devices.

3.4 Technology Foundation

The application is built on industry-standard front-end technologies:

- **Frontend:** HTML5, CSS3 (custom stylesheet), vanilla JavaScript
- **Icons:** Font Awesome Pro icon library
- **Component Loading:** Fetch API for dynamic navbar and footer injection

- **Hosting:** Compatible with any static web hosting environment (e.g. GitHub Pages, Netlify, Vercel)

3.5 Objectives

Primary Objective:

Build a fully responsive, multi-page front-end website that presents Upskill Hub's course catalog, community, and brand in a clean, professional, and easy-to-navigate manner.

Secondary Objectives:

- Implement reusable navigation and footer components shared across all pages
- Create a mobile-first responsive layout that adapts smoothly across screen sizes
- Design an engaging course-browsing experience with ratings, pricing, and update timestamps
- Lay a scalable front-end foundation for future backend and database integration

3.6 Future Scope

- **Backend & Database Integration:** Introduce a Node.js/Express backend with a database (MongoDB or PostgreSQL) to persist courses, users, and enrollments
- **User Authentication:** Add sign-up/login functionality so learners can track enrolled courses and progress
- **Payment Gateway Integration:** Enable secure course purchases through a payment gateway such as Razorpay or Stripe
- **Course Content Delivery:** Add video lecture streaming, downloadable resources, and quizzes within each course
- **Admin Panel:** Build a management dashboard for adding/editing courses, blog posts, and expert profiles
- **Search & Filtering:** Introduce course search, category filters, and sorting by price or rating

➤ Analysis Phase

4.1 Requirement Analysis

Functional Requirements

- FR1: System shall display available courses with thumbnail, title, rating, and price
- FR2: System shall provide consistent navigation (Home, About, Blog, Courses, Contact) across all pages
- FR3: System shall dynamically load the navbar and footer components using the Fetch API
- FR4: System shall provide a registration form capturing name, email, and phone number
- FR5: System shall display community expert profiles with social media links
- FR6: System shall provide a contact page with organization details and a contact form

Non-Functional Requirements

- NFR1: Page load time shall not exceed 3 seconds on 4G connection
- NFR2: Mobile navigation menu shall toggle smoothly without layout shift
- NFR3: Application shall be fully responsive across mobile, tablet, and desktop
- NFR4: Component-loading failures (navbar/footer fetch) shall be handled gracefully with console error logging
- NFR5: All static assets shall be optimized for fast rendering
- NFR6: UI shall maintain consistent styling and branding across every page

4.2 DFD and System Flow

Page & Component Load Flow :

- User navigates to any page of Upskill Hub (Home, About, Blog, Courses, or Contact)
- Browser loads the requested HTML page and linked stylesheet
- JavaScript fetches components/nav.html and injects it into the #navbar container
- JavaScript fetches components/footer.html and injects it into the #footer container
- Mobile menu event listeners are attached once the navbar is loaded
- User browses courses, expert profiles, or blog content rendered directly in the page
- User may submit the registration form or contact form to express interest
- Any fetch failure is caught and logged to the console without breaking the page

➤ Technology Stack (Web Development)

- **Frontend:** HTML5, CSS3, JavaScript
- **Icon Library:** Font Awesome Pro
- **Tools:** GitHub, VS Code, Figma.
- **Deployment:** GitHub Pages / Netlify.

➤ System Design & Methodology

6.1 Development Methodology

The project follows an iterative, incremental development approach:

- **Phase 1:** Requirement Analysis - Define target learners and core site sections
- **Phase 2:** UI/UX Design - Create wireframes and a responsive layout plan in Figma
- **Phase 3:** HTML Structuring - Build semantic markup for Home, About, Blog, Courses, and Contact pages

- **Phase 4:** CSS Styling - Implement a custom, responsive stylesheet with flexbox/grid layouts
- **Phase 5:** Component Integration - Add JavaScript Fetch-based navbar/footer loading and mobile menu logic
- **Phase 6:** Testing & Deployment - Cross-device testing and deployment to static hosting

6.2 Architecture Design

Frontend: A multi-page static site (index.html plus dedicated pages under /Navs) sharing a single stylesheet and a common image asset library. Components: The navigation bar and footer live as standalone HTML fragments inside /components and are dynamically fetched and injected into every page at runtime, ensuring a single source of truth for site-wide navigation and branding.

➤ Expected Outcomes

- A responsive, multi-device compatible course-discovery website.
- Consistent, reusable navigation and footer components across all pages.
- Deployment-ready static site hosted on a live server.

➤ References

- Mozilla Developer Network. Web APIs - Fetch. Retrieved from https://developer.mozilla.org/
- Font Awesome Official Documentation. Retrieved from https://fontawesome.com/docs
- W3C. HTML5 Specification. Retrieved from https://www.w3.org/TR/html52/
- MDN Web Docs. CSS Flexbox and Grid Layout. Retrieved from https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Flexible_Box_Layout

- GitHub Pages Documentation. Retrieved from https://docs.github.com/en/pages
- Google Web Fundamentals. Responsive Web Design Basics. Retrieved from https://web.dev/